

CUDA 并行计算与 GPGPU 编程架构速查表

L0 Essence: GPU 并行计算的本质是通过超大规模轻量级线程（Threads）在三维网格（Grid-Block-Thread）拓扑下，利用 SIMT（单指令多线程）硬件架构与高带宽显存（HBM/GDDR）实现高吞吐的数据并行吞吐与张量计算加速。

[M1.1] Grid-Block-Thread 层次模型

核心定义: CUDA 软件逻辑将线程划分为三维网格（Grid）和线程块（Block）。每个 Block 包含最多 1024 个线程，Block 内部线程可共享内存并进行屏障同步。
底层机理: 线程在物理上通过三维索引确定自身空间位置。一维索引计算公式为: $int\ tid = blockIdx.x * blockDim.x + threadIdx.x$ ；如果是二维网格，对应平面映射计算公式为: $int\ x = blockIdx.x * blockDim.x + threadIdx.x$ ； $int\ y = blockIdx.y * blockDim.y + threadIdx.y$ 。这构成了 GPU 软件并行调度的基础层。

[M1.2] Kernel 设备端运行生命周期

核心定义: 核函数（Kernel）是 CPU 发起、GPU 执行的异步并行执行函数。
底层机理: 主机端 CPU 线程调用 Kernel 时，通过执行配置参数 `<<<grid, block, sharedMem, stream>>>` 将硬件启动请求推入指定的 CUDA 流（Stream）任务队列中。CPU 随即返回继续执行后续代码，而 GPU 硬件驱动接收任务后异步分配网格进行网格发射（Grid Launch），这体现了主机/设备异构协同的非阻塞管道设计。

[M1.3] 流多处理器 (SM) 物理微架构

核心定义: 流多处理器（Streaming Multiprocessor, SM）是 GPU 的核心物理计算单元。
底层机理: 每个 SM 内部包含数十个 SP（Single Precision / CUDA Cores）、少量 DP（Double Precision Cores）、SFU（Special Function Units，执行 sine/cosine 等数学计算）以及超大规模的通用寄存器堆（Register File）。SM 每次从硬件 Warp 调度器中取出一个 Warp 并在硬件 Cores 上进行多路指令发射。

[M1.4] SIMT 架构与 Warp 调度逻辑

核心定义: 单指令多线程（Single Instruction Multiple Threads, SIMT）是 GPU 执行线程的核心硬件机制。
底层机理: 物理上 GPU 并不以单个线程为基本单元进行指令派发，而是将 32 个连续线程绑定为一个线程束（Warp）。Warp 内的 32 个线程共享同一个程序计数器（PC），在相同的时钟周期执行同一条指令。Warp 调度器每个周期在就绪的 Warp 队列中快速选择进行切换发射。

[M1.5] Warp Divergence (分支分歧) 规避

核心定义: 当同一个 Warp 内部的 32 个线程在执行条件分支时走入不同的分支路径，就会发生分支分歧。
底层机理: SIMT 架构下 32 个线程必须执行相同指令。若分支分歧，硬件将使用活动掩码（Active Mask）使一部分线程失活，然后串行化执行 `if` 和 `else` 分支路径。这导致执行吞吐减半。规避方法是使分支判定边界对齐为 32 的整数倍，避免在 Warp 内部发生数据分裂。

[M2.1] GPU 延迟隐藏与占用率 (Occupancy)

核心定义: 占用率是每个 SM 上实际活跃的 Warp 数量与硬件理论最大活跃 Warp 数量的比值。
底层机理: GPU 没有类似 CPU 的复杂分支预测或超大 L3 缓存。它通过快速上下文切换（Context Switch）来隐藏访问延迟。当某个 Warp 因读取全局显存被阻塞时，SM 内的 Warp 调度器会零开销切换至另一个已就绪的 Warp 执行。高 Occupancy 意味着有足够多的备用就绪 Warp，从而能够完美隐藏延迟。

[M2.2] GPU 物理存储分级层次

核心定义: GPU 的存储层次由寄存器、局部内存、共享内存、常量内存、纹理内存、L1/L2 缓存和全局显存构成。
底层机理: 线程访问寄存器（Register File）仅需 1 个时钟周期；访问片上共享内存（Shared Memory）约需数十个周期；而访问全局显存（Global Memory）则需高达 400 - 800 个时钟周期。高性能编程的核心是将数据从高延迟的全局显存搬运至高速片上 SRAM（Shared Memory）中复用。

[M2.3] Global Memory 合并访存 (Coalesced Access)

核心定义: 当一个 Warp 内部的 32 个线程同时访问全局显存时，硬件能够将这 32 个访存请求合并为一笔或几笔物理内存交易。
底层机理: 显存控制器以 32 字节、64 字节或 128 字节粒度进行内存段（Segment）对齐抓取。如果 32 个线程访问的地址是连续且对齐的，仅需 1 次内存读取事务。若访存是不连续或乱序的，显存控制器会被迫发起 32 次独立的内存事务，导致显存物理总带宽被极度稀释。

[M2.4] Shared Memory 与 32-Bank 组织

核心定义: 共享内存是位于流多处理器 SM 内部的超低延迟片上 SRAM，由 32 个划分为独立地址空间的 Bank 构成。
底层机理: 连续的 32-bit（或 64-bit）数据会被循环映射到 Bank 0 至 Bank 31。当一个 Warp 的 32 个线程在同一周期分别访问 32 个不同的 Bank 时，访存可以完全并行。Bank 架构保证了每个周期单 Block 能够提供数太字节的吞吐潜力。

[M2.5] Shared Memory Bank Conflict (银行冲突)

核心定义: 当同一个 Warp 内的两个或多个线程同时访问同一个 Bank 内部不同的地址时，会发生 Bank 冲突。
底层机理: 访问相同 Bank 的请求必须在硬件上被串行化（n-way 冲突会导致执行时间翻 n 倍）。特殊例外: 如果 Warp 内的所有线程同时访问同一个 Bank 内部的 `threadf` 相同地址，硬件会触发广播模式（Broadcast），这不视为冲突且仅占用 1 个时钟周期。规避方法是设计 Stride 步长为奇数，确保物理 Bank 索引完美错开。

[M3.1] 寄存器溢出 (Register Spilling) 的代价

核心定义: 当单个设备端线程申请的局部变量/数组过多，超过 SM 分配给每个线程的硬件寄存器配额上限时，会发生寄存器溢出。
底层机理: 编译器被迫将超出部分的变量移至局部内存（Local Memory）。局部内存存在物理上与全局显存（Global Memory）处于同一层高延迟介质中（受 L1 缓存保护，但依然比寄存器慢数百倍）。这会导致计算算子的指令吞吐暴跌。

L1 Logical Framework

网络拓扑映射: 软件逻辑层将计算任务抽象为 Grid 和 Block，由千万级轻量级线程在三维坐标系下通过 `threadIdx` 与 `blockIdx` 确定物理数据的一维/二维映射。
SIMT 硬件调度: 硬件执行层将线程块 Block 映射到流多处理器 SM（Streaming Multiprocessor），以 32 个线程的线程束 Warp 为基本物理调度单元在 SIMT 架构下执行。
高吞吐存储层级: 构建由线程私有寄存器、Block 共享内存（Shared Memory）、全局显存（Global Memory）构成的多级高吞吐存储层次，并使用合并访存（Coalesced Memory Access）与共享内存无 Bank 冲突（Bank Conflict）来最大化有效带宽。
异步流水与并发: 通过 `__syncthreads()` 执行 Block 内线程同步，利用流（Stream）与图（Graphs）技术实现异构 CPU-GPU 数据拷贝与计算在时间上的重叠（Overlap）并发。

L2 Data Flow Topology

主存数据 → (PCIe / NVLink 拷贝 HtoD) → GPU 全局显存 (Global Memory) → (网络加载 Grid Launch) → SM 缓存 / 寄存器 → (Shared Memory 协同块内 Reduction) → Warp 寄存器进行算术计算 (FMA / Tensor Core) → (Shared Memory / Global Memory 写回) → (PCIe / NVLink 拷贝 DtoH) → CPU 主存结果

Trade-off Matrix: Pipeline Overhead & Accuracy

访存带宽: 低至中等 (100 - 300 GB/s) → 高 (1.5 - 2.5 TB/s) [极高 (寄存器内直接交换, 无总线延迟)]
片上内存开销: 0 字节 (无需显式申请片上 SRAM) → 16KB - 96KB / Block [0 字节 (直接复用硬件通用寄存器 RF)]
同步开销: 无 (线程独立执行) → 中等 (需调用 `__syncthreads()` 屏障) [极低 (仅 Warp 内 32 线程同步)]
开发复杂度: 极低 (简单一维/多维指针索引) → 中高 (需手动划分子块、协作搬运、同步) [高 (需利用位移操作、掩码及 Warp 内对齐)]

[M3.3] L1 / L2 硬件级缓存调优

\\textbullet\\textbf 核心定义: 每个 SM 内的 L1 缓存和 Shared Memory 共享同一物理片上 SRAM, 可动态分配大小比例。\\textbullet\\textbf 底层机理: 开发者可通过 API 将 128KB 物理 SRAM 划分为 96KB Shared / 32KB L1 或 64KB Shared / 64KB L1。L2 缓存是全局的, 连接显存控制器, 可以支持非合并访存的局部对齐合并, 从而缓解环访存对显存带宽的冲击。

[M3.4] 统一内存 (Unified Memory) 页面迁移

\\textbullet\\textbf 核心定义: 统一内存通过统一虚拟地址空间 (UVA) 使得 CPU 和 GPU 共享相同的内存指针, 数据在两端自动透明流转。\\textbullet\\textbf 底层机理: 当 GPU 访问某个尚未迁移至显存的 UM 地址时, 触发 GPU 硬件页表缺页中断 (Page Fault)。硬件驱动通过 PCIe/NVLink 自动将对应的物理页面 (4KB 粒度) 异步搬运至 GPU 显存, 实现数据自动冷热流转。

[M3.5] 屏障同步与内存栅栏 (Barrier & Fence)

\\textbullet\\textbf 核心定义: 同步机制用于控制不同线程的执行顺序和内存数据的可见性。\\textbullet\\textbf 底层机理: __syncthreads() 执行执行流栅栏, 确保 Block 内所有线程均到达此指令。__threadfence() 执行内存栅栏, 它不阻塞执行流, 但强制冲刷所有挂起的写数据事务, 确保该指令前的写入对其他线程 (根据作用域不同, 如 Block、Grid、System) 立即可见。

[M4.1] 协作组 (Cooperative Groups) 通信机制

\\textbullet\\textbf 核心定义: 协作组是现代 CUDA 引入的灵活线程协同抽象, 打破了传统 __syncthreads() 必须在 Block 内全体同步的硬性限制。\\textbullet\\textbf 底层机理: 允许开发者在代码中定义任意子组 (Sub-groups, 如 4/8/16 线程), 并调用组内专属屏障或集体通信。通过 this_grid() 还可以跨越 Block 限制执行网格级别的同步, 实现无 CPU 干预的安全动态迭代计算。

[M4.2] Warp Shuffle (洗牌) 高速片上交换

\\textbullet\\textbf 核心定义: Warp Shuffle 是指 Warp 内部的线程可以直接读取同组内其他线程的寄存器值, 无需借助共享内存进行中转。\\textbullet\\textbf 底层机理: 硬件上通过特定的 __shfl_sync 或 __shfl_down_sync 原语直接在 SM 的 Register File 内部进行数据交叉总线中转, 避免了 Shared Memory 的访存开销。L1 缓存开销以及显式同步屏障, 是编写极致性能 Parallel Reduction 等算子的核心武器。

[M4.3] GPU 原子操作与硬件 ALU 支持

\\textbullet\\textbf 核心定义: 原子操作 (如 atomicAdd) 保证多个线程对同一全局或共享内存地址的写操作在硬件上是互斥且不可分割的。\\textbullet\\textbf 底层机理: 现代 GPU 的 L2 缓存控制器处集成了硬件原子 ALU。当对全局内存进行 atomicAdd 时, 访存包直接发送至 L2 并由硬件 ALU 在就地修改后写回, 完全不需要回传寄存器, 极大地缓解了高并发写入时的冲突锁自旋延迟。

[M4.4] CUDA Streams (并发流) 异步控制

\\textbullet\\textbf 核心定义: CUDA 流是代表一组在 GPU 上按顺序执行的异步任务队列。\\textbullet\\textbf 底层机理: 默认流 (Stream 0) 是同步阻塞的, 其上的任何操作都会隐式阻塞所有其他流的任务执行。非零流 (Non-blocking Streams) 则互不干扰。这允许我们在 GPU 硬件中实现真正的任务级并行, 是多 Kernel 重叠执行的控制中枢。

[M5.1] CPU-GPU 异步操作与 Event 监控

\\textbullet\\textbf 核心定义: 主机端启动 GPU 任务是异步的, Event 用于进行精确的时序度量 and 同步。\\textbullet\\textbf 底层机理: cudaEventRecord 将一个事件标记插入指定的流中。当 GPU 执行到该标记时, Event 的状态被更新为完成。主机端可通过 cudaEventQuery 查询事件状态, 或者调用 cudaEventSynchronize 强制挂起 CPU, 等待 GPU 流水线清空。

[M5.2] CUDA Graphs 静态执行拓朴

\\textbullet\\textbf 核心定义: CUDA Graphs 通过将多个 Kernel 启动和内存拷贝任务定义为静态 DAG 计算图, 实现一键提交。\\textbullet\\textbf 底层机理: 传统的多核函数调用会在 Host 端 (CPU) 产生大量的 API 调用和调度开销 (启动延迟约数微秒)。通过捕获机制生成的 Graph, 一经实例化并提交 (cudaGraphLaunch), 驱动会直接将整个拓朴一次性压入 GPU 硬件执行队列, 消除了 CPU-GPU 往返开销。

[M5.3] Page-locked (Pinned) 内存与 DMA 传输

\\textbullet\\textbf 核心定义: Pinned 内存是分配在 CPU 主存中、不允许操作系统进行虚拟内存页面交换 (Page Swap) 的物理连续内存。\\textbullet\\textbf 底层机理: 在标准的 malloc 内存拷贝中, 系统必须先将要数据拷贝到临时缓冲区中再发起传输。而使用 cudaHostAlloc 申请的 Pinned Memory 直接被物理锁定。GPU 的 DMA 硬件控制器可以直接读取该地址, 跳过 CPU 中转, 从而使 PCIe 物理总线的带宽达到上限。

[M5.4] Multi-GPU P2P 直接互联与 NVLink

\\textbullet\\textbf 核心定义: Peer-to-Peer (P2P) 允许同一主机下的多张显卡在不通过 CPU 主存中转的情况下直接读取彼此的显存。\\textbullet\\textbf 底层机理: 通过 NVLink/NVSwitch 高带宽物理网格线, 两张 GPU 可以绕过主板 PCIe 插槽和 CPU, 直接进行硬件级高带宽跨显卡读写, 延迟极低, 是当今超大规模 AI 大模型分布式训练的基础网络架构。

[M5.5] Tensor Cores 深度学习硬加速

\\textbullet\\textbf 核心定义: Tensor Cores 是 GPU 专门用于加速矩阵乘累加运算的定制化硬件计算单元。\\textbullet\\textbf 底层机理: 每个 Tensor Core 可以在硬件电路层面一步完成 $D = A \times B + C$ 的小矩阵乘法 (例如 16x16x16 规模)。它通过硬编码算术阵列并行计算大量乘法积, 极大地超越了普通 SP 核心的 FMA 执行速度。

[M6.1] CUDA 动态并行 (Dynamic Parallelism)

\\textbullet\\textbf 核心定义: 动态并行允许在设备端执行的 Kernel 直接发射启动其他的子核函数 (Child Kernels), 无需通过 CPU 回传及发起。\\textbullet\\textbf 底层机理: 硬件上 GPU 内置的微调度器 (Device-side Scheduler) 负责就地解析子核函数的发射配置并压入硬件队列。这允许实现极高效率的递归树形计算、稀疏流体计算及自适应网格细化。

Zone T1: CUDA APIs & Primitive Keywords

: 核心 API 函数与 Kernel 语法速查手册

Zone T2: Errors & Hardware Stall Dictionary

: 运行期异常与设备故障诊断字典

[M6.2] GPU 性能优化黄金法则

\\textbullet\\textbf 核心定义: 分辨并解决 Memory-bound (访存受限) 或 Compute-bound (计算受限) 这两种性能瓶颈的策略。\\textbullet\\textbf 底层机理: 使用 Roofline 模型对比算子的算术强度 (每字节传输的计算次数)。若受限于访存, 则重心在于提高 Global 合并率及 Shared 空间重用; 若受限于计算, 则重心在于通过指令循环展开、使用 fast-math 单元以及将 FMA 操作迁移至 Tensor Cores 提升吞吐。

[M6.3] GPGPU 经典并行算子设计模式

\\textbullet\\textbf 核心定义: 包含并行规约 (Reduction)、前缀和 (Scan)、直方图 (Histogram) 在内的基础算子设计。\\textbullet\\textbf 底层机理: 在并行规约中, 利用树形收敛以及最后 Warp 级的 Unrolled 循环, 避免不必要的 Block 栅栏同步; 在直方图计算中, 使用共享内存内部的 atomicAdd 缓存以规避全局显存的高冲突原子竞争。

[M6.4] Nsight Compute & Systems 诊断体系

\\textbullet\\textbf 核心定义: NVIDIA 官方的系统级和微架构级性能剖析分析工具链。\\textbullet\\textbf 底层机理: Nsight Systems 提供全局 Timeline, 用于观察 CPU-GPU Streams 拷贝与计算重叠度以及 API 延迟; Nsight Compute 对单个 Kernel 收集细粒度 PM (性能监视器) 计数器, 展示 SM 占用率、SOL (光速指标) 以及执行流阻塞的根本原因。